Práctica 1

Sistemas Operativos 2026/2027

Gestión de procesos y archivos. Comunicación entre procesos: tuberías y memoria compartida

Memoria de la práctica

Álvaro Penalva Castillo

Octubre de 2026

Índice general

1. GitHub **4**

2. Ejercicio 1 a): malla.c **5**

2.1. Análisis del enunciado 5

2.2. Planteamiento de la solución 5

2.3. Descripción del código 5

2.3.1. Función al_despertar() 5

2.3.2. Función rama(): última fila 6

2.3.3. Función rama(): creación del hijo y espera 6

2.3.4. Función main(): comprobación de argumentos y creación de las columnas 6

2.3.5. Función main(): espera final y mensaje de error 7

2.4. Compilación y comprobación 7

3. Ejercicio 1 b): ejec.c **8**

3.1. Análisis del enunciado 8

3.2. Planteamiento de la solución 8

3.3. Descripción del código 9

3.3.1. Variables globales 9

3.3.2. Función leer_argumentos() 9

3.3.3. Función main() 9

3.3.4. Función esperar_hijo() 10

3.3.5. Funciones crear_A() y proceso_A() 10

3.3.6. Funciones crear_B() y proceso_B() 11

3.3.7. Funciones manejador_aviso() y esperar_aviso() 12

3.3.8. Funciones crear_hijo_B(), proceso_hijo_B() y presentar_hijo_B() 12

3.3.9. Funciones crear_Z(), proceso_Z() y manejador_alarma_Z() 13

3.3.10. Funciones ejecutar_pstree() y manejador_tarea_A() 13

3.3.11. Función manejador_destruccion() 14

3.3.12. Función terminar_hijo() 14

3.3.13. Función manejador_destruir_A() 14

3.3.14. Función manejador_destruir_B() 15

3.3.15. Función manejador_destruir_hijo_B() 15

3.4. Compilación y ejecución 15

4. Ejercicio 2: hacha.c **16**

4.1. Análisis del enunciado 16

4.2. Planteamiento de la solución 16

4.3. Descripción del código 16

4.3.1. Funciones escribir_texto() y mostrar_uso() 16

4.3.2. Función main() 17

4.3.3. Función calcular_trozos() 17

4.3.4. Función crear_nombre_trozo() 18

4.3.5. Función partir(): apertura del archivo 18

4.3.6. Función partir(): creación de los hijos 18

4.3.7. Función enviar_trozo() 19

4.3.8. Función guardar_trozo() 19

4.3.9. Función partir(): espera final 20

4.4. Compilación y ejecución 20

5. Ejercicio 3: hijos.c..... 21
5.1. Análisis del enunciado..... 21
5.2. Planteamiento de la solución..... 21
5.3. Descripción del código..... 21
5.3.1. Variables globales..... 21
5.3.2. Función crear_memoria()..... 22
5.3.3. Función liberar_memoria()..... 22
5.3.4. Función crear_verticales()..... 22
5.3.5. Función crear_finales()..... 23
5.3.6. Funciones despierta() y esperar_alarma()..... 23
5.3.7. Función mostrar_pids()..... 24
5.3.8. Funciones mostrar_subhijo() y mostrar_superpadre()..... 24
5.3.9. Función main()..... 25
5.4. Compilación y ejecución..... 25

1. GitHub

Enlace al repositorio donde se encuentran los códigos fuente de la práctica (`malla.c`, `ejec.c`, `hacha.c` y `hijos.c`):

https://github.com/alvarocast5/SISTEMAS-OPERATIVOS.git

2. Ejercicio 1 a): malla.c

2.1. Análisis del enunciado

El enunciado pide un programa llamado `malla.c` que reciba dos argumentos, `x` e `y`, que representan el número de filas y de columnas de un árbol de procesos. El proceso inicial, `malla`, tiene `y` hijos (p11, p12, ..., p1y), uno por cada columna. Cada uno de esos hijos tiene a su vez un único hijo, y este otro, y así hasta completar `x` filas. En total hay `y` columnas con `x` procesos cada una, es decir, `x · y` procesos además del proceso inicial.

Para comprobar que la estructura creada es la correcta, el enunciado indica que se debe utilizar la orden `pstree -c`. Por tanto, los procesos tienen que permanecer vivos durante unos segundos. Las condiciones que debe cumplir el programa son las siguientes:

- Recibir dos argumentos que sean números mayores que 0.
- Crear exactamente el árbol indicado: `y` columnas de `x` procesos.
- Mantener los procesos vivos el tiempo suficiente para poder ejecutar `pstree -c`.
- Controlar la destrucción del árbol: ningún padre puede morir antes que sus hijos.
- Esperar sin utilizar `sleep`, empleando una alarma y `pause`.

2.2. Planteamiento de la solución

El problema se ha dividido en dos partes. En la primera, el proceso inicial utiliza un bucle para crear las `y` columnas: en cada vuelta crea un hijo con `fork` y ese hijo se encarga de construir su propia columna. En la segunda, la construcción de una columna se hace con una función llamada `rama`. Cada proceso de la columna crea un único hijo, que será el proceso de la fila siguiente, y la función se vuelve a llamar a sí misma en ese hijo hasta llegar a la última fila.

El proceso de la última fila no tiene hijos. Programa una alarma de 10 segundos y se queda parado con `pause`, lo que da tiempo a ejecutar `pstree -c`. Cuando suena la alarma, termina. Mientras tanto, el resto de procesos están esperando a su hijo con `wait`, de forma que mueren uno tras otro de abajo hacia arriba. El proceso inicial espera a todas las columnas y es el último en terminar.

2.3. Descripción del código

2.3.1. Función al_despertar()

Qué se necesita. Los procesos de la última fila deben esperar unos segundos sin utilizar `sleep`. Para ello se programa una alarma, que envía la señal SIGALRM cuando termina el tiempo. Según la práctica, si un proceso recibe una señal para la que no tiene manejador, muere. Por tanto, hace falta un manejador que permita que la señal despierte al proceso sin matarlo.

Cómo se ha resuelto y por qué. Se ha creado una función vacía, porque lo único que interesa es que la llegada de la señal haga que el proceso continúe. La instrucción `(void)sig;` solo indica al compilador que el parámetro no se utiliza y no influye en el funcionamiento.

```
static void al_despertar(int sig){
    (void)sig;
}
```

2.3.2. Función rama(): última fila

Qué se necesita. El proceso que se encuentra en la última fila no debe crear más hijos. Tiene que permanecer vivo unos segundos y después terminar.

Cómo se ha resuelto y por qué. La función recibe el nivel en el que está el proceso y el número total de filas. Cuando ambos valores coinciden, el proceso programa el manejador con `signal`, pone una alarma de 10 segundos con `alarm` y se detiene con `pause` hasta que llega la señal. Se han elegido 10 segundos para dejar tiempo suficiente a escribir `pstree -c` en la terminal. Después termina con `exit`.

```
if(nivel == filas){
    signal(SIGALRM, al_despertar);
    alarm(10);
    pause();
    exit(0);
}
```

Llamadas al sistema utilizadas: `signal(), alarm(), pause(), exit().`

2.3.3. Función rama(): creación del hijo y espera

Qué se necesita. Si el proceso no está en la última fila, tiene que crear a su hijo, que será el proceso de la fila siguiente, y no puede morir antes que él.

Cómo se ha resuelto y por qué. Se crea el hijo con `fork` y, si falla, se muestra un error. El hijo vuelve a llamar a `rama` con el nivel aumentado en uno, de modo que cada proceso crea al siguiente hasta llegar a la última fila. El padre espera con `wait` a que su hijo termine y solo entonces termina él con `exit`. Así, los hijos mueren siempre antes que los padres. Como `rama` siempre acaba con `exit`, el hijo nunca vuelve a la función que lo llamó.

```
pid = fork();
if(pid < 0){
    perror("Error en fork");
    exit(1);
}
if(pid == 0){
    rama(nivel + 1, filas);
}
wait(NULL);
exit(0);
```

Llamadas al sistema utilizadas: `fork(), wait(), exit().`

2.3.4. Función main(): comprobación de argumentos y creación de las columnas

Qué se necesita. El programa debe comprobar que recibe las filas y las columnas, que ambas son mayores que 0, y crear las `y` columnas a partir del proceso inicial.

Cómo se ha resuelto y por qué. Un `if` comprueba que hay dos argumentos y que, convertidos a número con `atoi`, ambos son mayores que 0. Después, un bucle `for` se repite una vez por cada columna. En cada vuelta, el proceso inicial hace un `fork`: el hijo creado llama a `rama` empezando en el nivel 1, mientras que el proceso inicial continúa con la vuelta siguiente para crear la columna siguiente.

```
    if(argc == 3 && atoi(argv[1]) > 0 && atoi(argv[2]) > 0){
        filas = atoi(argv[1]);
        columnas = atoi(argv[2]);

        for(c = 0; c < columnas; c++){
            pid = fork();
            if(pid < 0){
                perror("Error en fork");
                exit(1);
            }
            if(pid == 0){
                rama(1, filas);
            }
        }
```

Llamadas al sistema utilizadas: `fork()`.

2.3.5. Función main(): espera final y mensaje de error

Qué se necesita. El proceso inicial no puede terminar antes que sus hijos. Además, si los argumentos no son correctos hay que avisar al usuario.

Cómo se ha resuelto y por qué. Un segundo bucle realiza un `wait` por cada columna creada, de manera que el proceso inicial termina el último. Si los argumentos no son válidos se ejecuta la parte `else`, que muestra cómo se debe utilizar el programa.

```
        for(c = 0; c < columnas; c++){
            wait(NULL);
        }
    }
    else{
        printf("Uso: %s x(filas) y(columnas), ambos mayores que 0\n", argv[0]);
    }
```

Llamadas al sistema utilizadas: `wait()`.

2.4. Compilación y comprobación

El programa se compila y se ejecuta de la siguiente manera. Al ejecutarlo en segundo plano con `&`, se puede escribir `pstree -c` en la misma terminal antes de que pasen los 10 segundos:

```
$ gcc -o malla malla.c
$ ./malla 3 4 &
$ pstree -c
```

Con estos argumentos (3 filas y 4 columnas) debe verse el proceso `malla` con cuatro ramas, cada una formada por una cadena de tres procesos. Pasados los 10 segundos, todos los procesos terminan, empezando por los de la última fila.

3. Ejercicio 1 b): ejec.c

3.1. Análisis del enunciado

El enunciado pide un programa llamado `ejec.c` que reciba un argumento, los segundos de espera, y genere un árbol de procesos en el que `ejec` es el proceso inicial, A es su hijo, B es hijo de A y X, Y y Z son hijos de B. Además, debe realizar lo siguiente:

- Cada proceso muestra un mensaje con su PID y los PID de sus antecesores (padre, abuelo y bisabuelo, según corresponda).
- El proceso Z, pasados los segundos indicados, envía una señal al proceso A. Z no puede utilizar `sleep`, por lo que debe programar una alarma.
- El proceso A, al recibir la señal, ejecuta el comando `pstree`.
- Se debe controlar la destrucción del árbol: los padres no pueden morir antes que los hijos. El orden de muerte es Z, Y, X, B, A y ejec.

La salida que se espera por pantalla es la siguiente:

```
$ ejec 15
Soy el proceso ejec: mi pid es 751
Soy el proceso A: mi pid es 752. Mi padre es 751
Soy el proceso B: mi pid es 753. Mi padre es 752. Mi abuelo es 751
Soy el proceso X: mi pid es 754. Mi padre es 753. Mi abuelo es 752. Mi bisabuelo es 751
Soy el proceso Y: mi pid es 755. Mi padre es 753. Mi abuelo es 752. Mi bisabuelo es 751
Soy el proceso Z: mi pid es 756. Mi padre es 753. Mi abuelo es 752. Mi bisabuelo es 751
/* Tras un intervalo de 15 segundos aparecerá */
/* Resultado del comando pstree */
Soy Z (756) y muero
Soy Y (755) y muero
Soy X (754) y muero
Soy B (753) y muero
Soy A (752) y muero
Soy ejec (751) y muero
```

3.2. Planteamiento de la solución

Creación del árbol. Cada proceso crea a su hijo con `fork` y muestra su mensaje. Para que los mensajes salgan en el orden del enunciado, B no crea un hijo hasta que el anterior le ha avisado, mediante la señal SIGUSR1, de que ya se ha presentado.

Funcionalidad. Z programa una alarma con los segundos recibidos y se queda parado con `pause`. Cuando suena, envía SIGUSR1 a A. A, al recibir esa señal, ejecuta `pstree`. Como `exec` sustituye el programa del proceso que lo llama, A no puede ejecutarlo directamente: crea un hijo temporal que ejecuta `pstree` y A espera a que termine. Después, A avisa a `ejec` con la señal SIGUSR2 de que ya se puede destruir el árbol.

Destrucción. Cuando `ejec` recibe SIGUSR2, la reenvía a A. A la envía a B y espera a que B muera. B la envía primero a Z, espera a que muera, después a Y y por último a X, esperando en cada caso. De esta forma cada padre solo termina cuando su hijo ya ha muerto y el orden de muerte es Z, Y, X, B, A y `ejec`.

3.3. Descripción del código

3.3.1. Variables globales

Qué se necesita. Los procesos tienen que enviarse señales entre sí y, para ello, necesitan conocer el PID del proceso al que quieren avisar. También hay que guardar los segundos recibidos como argumento.

Cómo se ha resuelto y por qué. Se han declarado como variables globales los PID de cada proceso y los segundos. Además, `aviso_recibido` se utiliza para que B sepa que un hijo le ha avisado y `nombre_hijo_B` guarda el nombre de X, Y o Z para su mensaje de despedida. Cada proceso guarda su PID o el de su hijo nada más crearlo y, como `fork` copia las variables, los procesos que se crean después ya conocen los PID de sus antecesores. Por ejemplo, gracias a esto Z puede enviar una señal a A.

```
pid_t pid_ejec;
pid_t pid_A;
pid_t pid_B;
pid_t pid_X;
pid_t pid_Y;
pid_t pid_Z;

int segundos;
int aviso_recibido = 0;
char *nombre_hijo_B;
```

3.3.2. Función leer_argumentos()

Qué se necesita. El programa debe recibir un único argumento, los segundos, y este debe ser un número mayor que 0.

Cómo se ha resuelto y por qué. Se comprueba que hay dos elementos en la línea de órdenes (el nombre del programa y los segundos) y que el valor, convertido con `atoi`, es mayor que 0. Si no es así, se muestra cómo se debe usar el programa y se termina con `exit`. Si es correcto, se guarda en la variable `segundos`.

```
void leer_argumentos(int argc, char *argv[]) {
    if (argc != 2 || atoi(argv[1]) <= 0) {
        printf("Error. Uso: %s segundos (mayor que 0)\n", argv[0]);
        exit(1);
    }
    segundos = atoi(argv[1]);
}
```

Llamadas al sistema utilizadas: `exit()`.

3.3.3. Función main()

Qué se necesita. Mostrar el mensaje de `ejec`, dejar preparada la recepción de la orden de destrucción y crear el árbol.

Cómo se ha resuelto y por qué. Primero se leen los argumentos y se muestra el PID de `ejec`. Después se programa el manejador de SIGUSR2, que es la señal con la que A avisará de que ya ha ejecutado `pstree`. A continuación se crea A. Cuando A termina, `ejec` muestra su mensaje de despedida, por lo que es el último en morir.

```
int main(int argc, char *argv[]) {
    leer_argumentos(argc, argv);

    pid_ejec = getpid();
    printf("Soy el proceso ejec: mi pid es %d\n", pid_ejec);

    signal(SIGUSR2, manejador_destruccion);
    crear_A();

    printf("Soy ejec (%d) y muero\n", pid_ejec);
    return 0;
}
```

Llamadas al sistema utilizadas: `getpid()`, `signal()`.

3.3.4. Función esperar_hijo()

Qué se necesita. Cada padre debe esperar a que muera su hijo antes de terminar. Según la práctica, `wait` también termina si el proceso recibe una señal, y en este programa los padres reciben señales mientras esperan.

Cómo se ha resuelto y por qué. Se ha creado una función que repite `wait` hasta que el proceso que ha muerto es el hijo que se estaba esperando. Así, aunque una señal interrumpa la espera, el padre sigue esperando y nunca termina antes que su hijo.

```
void esperar_hijo(pid_t pid) {
    while (wait(NULL) != pid) {
    }
}
```

Llamadas al sistema utilizadas: `wait()`.

3.3.5. Funciones crear_A() y proceso_A()

Qué se necesita. Crear A como hijo de `ejec`, mostrar su mensaje y dejarlo preparado para recibir las señales.

Cómo se ha resuelto y por qué. `crear_A` hace el `fork`: el hijo ejecuta `proceso_A` y el padre (`ejec`) espera con `esperar_hijo`. `proceso_A` guarda su PID, muestra su mensaje y programa dos manejadores: SIGUSR1 (la orden de ejecutar `pstree`, que enviará Z) y SIGUSR2 (la orden de destrucción). Se programan antes de crear a B para que ninguna señal pueda llegar cuando todavía no existe su manejador. Por último, crea a B.

```
void crear_A(void) {
    pid_A = fork();

    if (pid_A == 0) {
        proceso_A();
        exit(0);
    }
    esperar_hijo(pid_A);
}
```

```
void proceso_A(void) {
    pid_A = getpid();
    printf("Soy el proceso A: mi pid es %d. Mi padre es %d\n", pid_A, pid_ejec);

    signal(SIGUSR1, manejador_tarea_A);
    signal(SIGUSR2, manejador_destruir_A);

    crear_B();
}
```

Llamadas al sistema utilizadas: `fork(), getpid(), signal(), exit().`

3.3.6. Funciones crear_B() y proceso_B()

Qué se necesita. Crear B, mostrar su mensaje y crear a sus tres hijos X, Y y Z haciendo que los mensajes salgan en el orden pedido.

Cómo se ha resuelto y por qué. `crear_B` funciona igual que `crear_A`. `proceso_B` muestra su mensaje, programa los manejadores de SIGUSR1 (aviso de los hijos) y de SIGUSR2 (destrucción) y crea los hijos uno a uno. Después de crear cada uno llama a `esperar_aviso`, que detiene a B hasta que ese hijo le avisa de que ya ha mostrado su mensaje. Así X, Y y Z se presentan siempre en ese orden. Al final, B se queda en un bucle con `pause` esperando la orden de destrucción.

```
void crear_B(void) {
    pid_B = fork();

    if (pid_B == 0) {
        proceso_B();
        exit(0);
    }
    esperar_hijo(pid_B);
}

void proceso_B(void) {
    pid_B = getpid();
    printf("Soy el proceso B: mi pid es %d. Mi padre es %d. Mi abuelo es %d\n",
           pid_B, pid_A, pid_ejec);

    signal(SIGUSR1, manejador_aviso);
    signal(SIGUSR2, manejador_destruir_B);

    pid_X = crear_hijo_B("X");
    esperar_aviso();
    pid_Y = crear_hijo_B("Y");
    esperar_aviso();
    pid_Z = crear_Z();
    esperar_aviso();

    while (1) {
        pause();
    }
}
```

Llamadas al sistema utilizadas: `fork(), signal(), pause().`

3.3.7. Funciones manejador_aviso() y esperar_aviso()

Qué se necesita. B tiene que detenerse hasta recibir el aviso de cada hijo. Sin embargo, la práctica explica que las señales no se recuerdan: si el aviso llegara justo antes de que B se pusiera a esperar, se perdería y B se quedaría parado para siempre.

Cómo se ha resuelto y por qué. El manejador anota en la variable `aviso_recibido` que ha llegado el aviso. La función `esperar_aviso` solo usa `pause` mientras la variable vale 0 y, al terminar, la vuelve a poner a 0 para el aviso siguiente. De este modo no se pierde ningún aviso aunque llegue antes de tiempo. Además, el manejador se vuelve a programar a sí mismo, porque según la práctica un manejador solo sirve una vez y B recibe tres avisos.

```
void manejador_aviso(int s) {
    signal(SIGUSR1, manejador_aviso);
    aviso_recibido = 1;
}

void esperar_aviso(void) {
    while (aviso_recibido == 0) {
        pause();
    }
    aviso_recibido = 0;
}
```

Llamadas al sistema utilizadas: `signal(), pause()`.

3.3.8. Funciones crear_hijo_B(), proceso_hijo_B() y presentar_hijo_B()

Qué se necesita. Crear X e Y, mostrar el mensaje con su padre, abuelo y bisabuelo, avisar a B y quedarse esperando la orden de destrucción.

Cómo se ha resuelto y por qué. `crear_hijo_B` recibe el nombre del proceso y hace el `fork`. En el hijo, `proceso_hijo_B` guarda su nombre (para el mensaje final), programa el manejador de destrucción, se presenta y se queda con `pause`. `presentar_hijo_B` muestra el mensaje con los PID del padre, el abuelo y el bisabuelo y después envía SIGUSR1 a B con `kill` como aviso. Al usar las mismas funciones para X y para Y, los mensajes son idénticos y solo cambia el nombre.

```
pid_t crear_hijo_B(char *nombre) {
    pid_t pid = fork();

    if (pid == 0) {
        proceso_hijo_B(nombre);
        exit(0);
    }
    return pid;
}

void proceso_hijo_B(char *nombre) {
    nombre_hijo_B = nombre;
    signal(SIGUSR2, manejador_destruir_hijo_B);
    presentar_hijo_B(nombre);

    while (1) {
        pause();
    }
}
```

```
void presentar_hijo_B(char *nombre) {
    printf("Soy el proceso %s: mi pid es %d. Mi padre es %d. Mi abuelo es %d. Mi
bisabuelo es %d\n",
           nombre, getpid(), pid_B, pid_A, pid_ejec);
    kill(pid_B, SIGUSR1);
}
```

Llamadas al sistema utilizadas: `fork(), signal(), kill(), pause(), getpid().`

3.3.9. Funciones crear_Z(), proceso_Z() y manejador_alarma_Z()

Qué se necesita. Z debe esperar los segundos indicados sin utilizar `sleep` y, pasado ese tiempo, enviar una señal a A.

Cómo se ha resuelto y por qué. `proceso_Z` es parecido a X e Y, pero además programa el manejador de SIGALRM. Se presenta y, a continuación, programa la alarma con `alarm(segundos)` y se queda esperando con `pause`. La alarma se programa después de presentarse para que los segundos se cuenten cuando el árbol ya está completo. Cuando termina el tiempo se ejecuta `manejador_alarma_Z`, que envía SIGUSR1 a A con `kill`.

```
pid_t crear_Z(void) {
    pid_t pid = fork();

    if (pid == 0) {
        proceso_Z();
        exit(0);
    }
    return pid;
}

void proceso_Z(void) {
    nombre_hijo_B = "Z";
    signal(SIGUSR2, manejador_destruir_hijo_B);
    signal(SIGALRM, manejador_alarma_Z);
    presentar_hijo_B("Z");

    alarm(segundos);

    while (1) {
        pause();
    }
}

void manejador_alarma_Z(int s) {
    kill(pid_A, SIGUSR1);
}
```

Llamadas al sistema utilizadas: `fork(), signal(), alarm(), pause(), kill().`

3.3.10. Funciones ejecutar_pstree() y manejador_tarea_A()

Qué se necesita. Cuando A recibe la señal de Z tiene que ejecutar `pstree` y después avisar a `ejec` de que ya puede comenzar la destrucción del árbol.

Cómo se ha resuelto y por qué. La práctica explica que `exec` sustituye el programa del proceso que lo llama. Si A ejecutara `pstree` directamente dejaría de ser A y se perdería el árbol. Por eso `ejecutar_pstree` crea un hijo con `fork`, que ejecuta `pstree` con `execlp`, mientras A espera

con `esperar_hijo` a que termine. Si `execlp` fallara, el hijo muestra un error y termina. Cuando `pstree` ha acabado, el manejador `manejador_tarea_A` envía SIGUSR2 a `ejec` para avisarle.

```
void ejecutar_pstree(void) {
    pid_t pid = fork();

    if (pid == 0) {
        execlp("pstree", "pstree", NULL);
        perror("Error al ejecutar pstree");
        exit(1);
    }
    esperar_hijo(pid);
}

void manejador_tarea_A(int s) {
    ejecutar_pstree();
    kill(pid_ejec, SIGUSR2);
}
```

Llamadas al sistema utilizadas: `fork(), execlp(), wait(), kill(), exit().`

3.3.11. Función manejador_destruccion()

Qué se necesita. Cuando `ejec` recibe el aviso de que A ha terminado su tarea tiene que iniciar la destrucción del árbol, pero sin morir antes que sus descendientes.

Cómo se ha resuelto y por qué. El manejador de `ejec` solo reenvía la señal SIGUSR2 a A. Mientras tanto, `ejec` sigue esperando dentro de `crear_A` y no termina hasta que A ha muerto.

```
void manejador_destruccion(int s) {
    kill(pid_A, SIGUSR2);
}
```

Llamadas al sistema utilizadas: `kill().`

3.3.12. Función terminar_hijo()

Qué se necesita. Cada padre tiene que ordenar la muerte de su hijo y comprobar que ha muerto antes de continuar.

Cómo se ha resuelto y por qué. La función envía SIGUSR2 al hijo con `kill` y espera con `esperar_hijo` a que muera. Se utiliza en A y en B para que la destrucción siempre se haga en el orden correcto.

```
void terminar_hijo(pid_t pid) {
    kill(pid, SIGUSR2);
    esperar_hijo(pid);
}
```

Llamadas al sistema utilizadas: `kill(), wait().`

3.3.13. Función manejador_destruir_A()

Qué se necesita. A debe transmitir la orden de destrucción a B y morir cuando B haya muerto.

Cómo se ha resuelto y por qué. El manejador llama a `terminar_hijo` con el PID de B y, cuando B ya ha muerto, muestra su mensaje de despedida y termina con `exit`.

```
void manejador_destruir_A(int s) {
    terminar_hijo(pid_B);
    printf("Soy A (%d) y muero\n", pid_A);
    exit(0);
}
```

Llamadas al sistema utilizadas: `exit()`.

3.3.14. Función manejador_destruir_B()

Qué se necesita. B tiene que destruir a sus hijos en el orden Z, Y, X y después morir.

Cómo se ha resuelto y por qué. El manejador llama tres veces a `terminar_hijo`: primero con Z, luego con Y y por último con X. Como cada llamada espera a que el hijo muera, no se pasa al siguiente hasta que el anterior ha terminado, y los mensajes salen en el orden pedido. Después B muestra su mensaje y termina.

```
void manejador_destruir_B(int s) {
    terminar_hijo(pid_Z);
    terminar_hijo(pid_Y);
    terminar_hijo(pid_X);
    printf("Soy B (%d) y muero\n", pid_B);
    exit(0);
}
```

Llamadas al sistema utilizadas: `exit()`.

3.3.15. Función manejador_destruir_hijo_B()

Qué se necesita. X, Y y Z deben morir al recibir la orden, mostrando su nombre y su PID.

Cómo se ha resuelto y por qué. El manejador muestra el mensaje con el nombre guardado y el PID del proceso y termina con `exit`. Es el mismo manejador para los tres procesos.

```
void manejador_destruir_hijo_B(int s) {
    printf("Soy %s (%d) y muero\n", nombre_hijo_B, getpid());
    exit(0);
}
```

Llamadas al sistema utilizadas: `getpid()`, `exit()`.

3.4. Compilación y ejecución

```
$ gcc -o ejec ejec.c
$ ./ejec 15
```

La salida por pantalla es la que se muestra en el análisis del enunciado: primero los mensajes de presentación de cada proceso, después, pasados los segundos indicados, el resultado de `pstree` y, por último, los mensajes de muerte en el orden Z, Y, X, B, A y ejec.

4. Ejercicio 2: hacha.c

4.1. Análisis del enunciado

El enunciado pide un programa llamado `hacha.c` que divida un archivo en varios trozos. Se ejecuta con `hacha <archivo> <tamaño>`, donde `<archivo>` es el nombre del archivo que se quiere dividir y `<tamaño>` es el tamaño en bytes de cada trozo. Los trozos tienen el mismo nombre que el archivo original con la extensión `.h00`, `.h01`, `.h02`, etc. Las condiciones que se deben cumplir son:

- El proceso padre crea tantos procesos hijos como trozos haya que crear.
- El padre envía la información a los hijos mediante tuberías, y son los hijos quienes crean los archivos de destino y escriben en ellos.
- Las entradas y salidas a archivos se realizan con llamadas al sistema vistas en la práctica. No se puede utilizar `printf`, `scanf`, etc.
- Es decisión del alumno si los hijos se lanzan uno detrás de otro o a la vez.

Ejemplo del enunciado:

```
$ hacha at_madrid.mp3 50000
$ ls
at_madrid.mp3.h00
at_madrid.mp3.h01
at_madrid.mp3.h02
```

4.2. Planteamiento de la solución

El padre abre el archivo y calcula cuántos trozos hay: divide el tamaño total entre el tamaño del trozo y, si sobran bytes, suma uno más, ya que el último trozo será más pequeño. Para cada trozo se crea una tubería y un hijo. El padre lee del archivo original tantos bytes como el tamaño del trozo y los escribe en la tubería; el hijo los lee de la tubería y los guarda en su archivo.

Se utiliza una tubería distinta para cada hijo porque, como explica la práctica, un proceso solo puede usar las tuberías que ha heredado de su padre, y de esta forma cada hijo recibe únicamente su trozo. Cada proceso cierra el extremo de la tubería que no utiliza. Esto es importante porque el hijo solo detecta que no van a llegar más datos (fin de archivo) cuando todos los extremos de escritura están cerrados.

Se ha decidido lanzar los hijos de forma concurrente: el padre crea un hijo, le envía su trozo y pasa al siguiente sin esperar a que termine. Cuando ha enviado todos los trozos, espera con `wait` a que terminen todos los hijos.

4.3. Descripción del código

4.3.1. Funciones escribir_texto() y mostrar_uso()

Qué se necesita. El programa tiene que mostrar mensajes por pantalla sin utilizar `printf`.

Cómo se ha resuelto y por qué. `escribir_texto` recibe un descriptor y un texto y lo escribe con `write`, calculando su longitud con `strlen`. Con el descriptor 1, que corresponde a la salida

estándar según la tabla de descriptores de la práctica, el texto aparece en pantalla. `mostrar_uso` utiliza esta función para explicar cómo se debe ejecutar el programa.

```
void escribir_texto(int descriptor, char *texto) {
    write(descriptor, texto, strlen(texto));
}

void mostrar_uso(char *programa) {
    escribir_texto(1, "Error. Uso: ");
    escribir_texto(1, programa);
    escribir_texto(1, " fichero tam_trozo\n");
}
```

Llamadas al sistema utilizadas: `write()`.

4.3.2. Función main()

Qué se necesita. Comprobar que se reciben los dos argumentos y que el tamaño del trozo es un número mayor que 0.

Cómo se ha resuelto y por qué. Si el número de argumentos no es el correcto o el tamaño convertido con `atoi` no es mayor que 0, se muestra un mensaje de error y se termina. Si todo es correcto, se llama a la función `partir`, que realiza el trabajo.

```
int main(int argc, char *argv[]) {
    int tamanyo;

    if (argc != 3) {
        mostrar_uso(argv[0]);
        return 1;
    }

    tamanyo = atoi(argv[2]);
    if (tamanyo <= 0) {
        escribir_texto(1, "Error. El tamanyo del trozo debe ser mayor que 0\n");
        return 1;
    }

    partir(argv[1], tamanyo);
    return 0;
}
```

4.3.3. Función calcular_trozos()

Qué se necesita. Hay que saber cuántos trozos se van a crear, ya que se necesita un hijo por cada uno.

Cómo se ha resuelto y por qué. La llamada `stat` proporciona información del archivo, entre ella su tamaño total en bytes (`st_size`). Este tamaño se divide entre el tamaño del trozo y, si la división no es exacta, se suma un trozo más para los bytes que sobran.

```
int calcular_trozos(char *nombre, int tamanyo) {
    struct stat propiedades;
    int trozos;

    stat(nombre, &propiedades);
    trozos = propiedades.st_size / tamanyo;
    if (propiedades.st_size % tamanyo != 0) {
        trozos++;
    }
    return trozos;
}
```

Llamadas al sistema utilizadas: `stat()`.

4.3.4. Función crear_nombre_trozo()

Qué se necesita. Cada trozo debe llamarse como el archivo original seguido de `.h` y un número de dos cifras (`.h00`, `.h01`, ...).

Cómo se ha resuelto y por qué. Se utiliza `sprintf` con el formato `%02d`, que escribe el número con dos cifras añadiendo un cero a la izquierda si hace falta. El resultado no se muestra por pantalla, sino que se guarda en una cadena que luego se usa para crear el archivo.

```
void crear_nombre_trozo(char *destino, char *nombre, int numero) {
    sprintf(destino, "%s.h%02d", nombre, numero);
}
```

4.3.5. Función partir(): apertura del archivo

Qué se necesita. Abrir el archivo original y avisar al usuario si no existe.

Cómo se ha resuelto y por qué. Se abre con `open` en modo solo lectura. Si devuelve un valor negativo significa que ha habido un error, por lo que se muestra un mensaje con `escribir_texto` y se termina la función sin crear ningún hijo.

```
void partir(char *nombre, int tamanyo) {
    int entrada, trozos, i;
    int tubo[2];
    char *buffer;
    pid_t pid;

    entrada = open(nombre, O_RDONLY);
    if (entrada < 0) {
        escribir_texto(1, "Error. No existe el fichero\n");
        return;
    }
```

Llamadas al sistema utilizadas: `open()`.

4.3.6. Función partir(): creación de los hijos

Qué se necesita. Crear una tubería y un hijo por cada trozo, y enviar a cada hijo su parte del archivo.

Cómo se ha resuelto y por qué. Primero se calcula el número de trozos y se reserva con `malloc` un espacio en memoria del tamaño de un trozo, que se utilizará para leer del archivo. Esta función no aparece en la práctica, pero es necesaria porque el tamaño lo decide el usuario al ejecutar el programa. Después, un bucle repite una vez por trozo: crea la tubería con `pipe` y el hijo con `fork`.

El hijo cierra el archivo original, que no necesita, y guarda su trozo. El padre continúa y envía el trozo a la tubería con `enviar_trozo`. Como el padre no espera después de crear cada hijo, los hijos trabajan a la vez.

```
    trozos = calcular_trozos(nombre, tamanyo);
    buffer = (char *) malloc(tamanyo);

    for (i = 0; i < trozos; i++) {
        pipe(tubo);
        pid = fork();
        if (pid == 0) {
            close(entrada);
            free(buffer);
            guardar_trozo(tubo, nombre, i, tamanyo);
            exit(0);
        }
        enviar_trozo(entrada, tubo, buffer, tamanyo);
    }
```

Llamadas al sistema utilizadas: `pipe(), fork(), close(), exit().`

4.3.7. Función enviar_trozo()

Qué se necesita. El padre tiene que leer un trozo del archivo original y enviárselo al hijo correspondiente.

Cómo se ha resuelto y por qué. El padre cierra el extremo de lectura de la tubería, que no va a usar, y lee del archivo con `read` como máximo el tamaño del trozo. Si ha leído algún dato, lo escribe en la tubería con `write`, indicando exactamente los bytes leídos, porque el último trozo puede ser más corto. Por último cierra el extremo de escritura, lo que avisa al hijo de que ya no llegarán más datos.

```
void enviar_trozo(int entrada, int tubo[2], char *buffer, int tamanyo) {
    int leidos;

    close(tubo[0]);
    leidos = read(entrada, buffer, tamanyo);
    if (leidos > 0) {
        write(tubo[1], buffer, leidos);
    }
    close(tubo[1]);
}
```

Llamadas al sistema utilizadas: `close(), read(), write().`

4.3.8. Función guardar_trozo()

Qué se necesita. Cada hijo tiene que recibir los datos por la tubería, crear el archivo del trozo y escribir en él.

Cómo se ha resuelto y por qué. El hijo cierra primero el extremo de escritura de la tubería, porque si lo dejara abierto nunca detectaría el fin de los datos. Después reserva su espacio de lectura, crea el nombre del trozo y el archivo con `creat` (con permisos 0666). A continuación lee de la tubería y escribe en el archivo mientras `read` devuelva datos; cuando devuelve 0 significa fin de archivo y el bucle termina. Finalmente cierra el archivo y la tubería y libera la memoria.

```
void guardar_trozo(int tubo[2], char *nombre, int numero, int tamanyo) {
    char nombre_trozo[256];
    char *trozo;
    int salida, leidos;

    close(tubo[1]);
    trozo = (char *) malloc(tamanyo);
    crear_nombre_trozo(nombre_trozo, nombre, numero);
    salida = creat(nombre_trozo, 0666);

    leidos = read(tubo[0], trozo, tamanyo);
    while (leidos > 0) {
        write(salida, trozo, leidos);
        leidos = read(tubo[0], trozo, tamanyo);
    }

    close(salida);
    close(tubo[0]);
    free(trozo);
}
```

Llamadas al sistema utilizadas: `close()`, `creat()`, `read()`, `write()`.

4.3.9. Función partir(): espera final

Qué se necesita. El padre no puede terminar antes de que todos los hijos hayan acabado de escribir sus archivos.

Cómo se ha resuelto y por qué. Un bucle realiza un `wait` por cada hijo creado. Después el padre cierra el archivo original y libera la memoria reservada.

```
    for (i = 0; i < trozos; i++) {
        wait(NULL);
    }

    close(entrada);
    free(buffer);
```

Llamadas al sistema utilizadas: `wait()`, `close()`.

4.4. Compilación y ejecución

```
$ gcc -o hacha hacha.c
$ ./hacha at_madrid.mp3 50000
```

Se crea un archivo por cada trozo de 50000 bytes (`at_madrid.mp3.h00`, `at_madrid.mp3.h01`, ...), tal y como muestra el ejemplo del enunciado.

5. Ejercicio 3: hijos.c

5.1. Análisis del enunciado

El enunciado pide un programa llamado `hijos.c` que cree un árbol de procesos a partir de dos parámetros, `x` e `y`, ejecutándolo como `hijos x y`. El proceso inicial (`hijos`) es el superpadre. De él cuelga una cadena de `x` procesos, cada uno hijo del anterior, y el último de la cadena tiene `y` hijos finales. Cada proceso debe mostrar un mensaje:

- El superpadre muestra: «Soy el superpadre (pid) : mis hijos finales son: pidx1, pidx2, ..., pidxy».
- Cada hijo final muestra: «Soy el subhijo pidx1, mi padres son: pid1, pid2, ..., pidx».

La dificultad del ejercicio está en que el superpadre debe mostrar los PID de procesos que no ha creado directamente, y cada hijo final debe mostrar los PID de todos sus antecesores. Como cada proceso trabaja con su propia copia de las variables, no es posible compartir esta información con variables normales. Por ello, el ejercicio se resuelve con memoria compartida, tal y como indica el título del apartado en la práctica.

5.2. Planteamiento de la solución

Se crean dos vectores de números enteros en memoria compartida. El primero, de tamaño `x`, guarda los PID de los procesos de la cadena vertical. El segundo, de tamaño `y`, guarda los PID de los hijos finales. La memoria se crea antes de crear ningún proceso para que todos la hereden con `fork`.

Cada proceso de la cadena guarda su PID en su posición del primer vector justo después de nacer y antes de crear a su hijo. De este modo, cuando nacen los hijos finales, el vector con los PID de sus padres ya está completo. Cada hijo final guarda su PID en el segundo vector, muestra su mensaje leyendo el primer vector y se queda esperando 10 segundos con una alarma, lo que permite comprobar el árbol con `pstree`.

Cuando los hijos finales terminan, cada proceso de la cadena va muriendo hacia arriba, ya que cada padre espera a su hijo con `wait`. Por último, el superpadre, que ya tiene completo el vector con los PID de los hijos finales, muestra su mensaje y libera la memoria compartida.

5.3. Descripción del código

5.3.1. Variables globales

Qué se necesita. Hay que guardar los identificadores de los dos segmentos de memoria compartida y las direcciones para acceder a ellos.

Cómo se ha resuelto y por qué. Se han declarado dos identificadores (`shmid_verticales` y `shmid_finales`) y dos punteros a enteros (`pids_verticales` y `pids_finales`). Son globales para que todas las funciones puedan acceder a los vectores y para que los procesos hijos los hereden.

```
int shmid_verticales, shmid_finales;
int *pids_verticales, *pids_finales;
```

5.3.2. Función crear_memoria()

Qué se necesita. Crear los dos vectores en memoria compartida antes de crear los procesos.

Cómo se ha resuelto y por qué. Para cada vector se llama a `shmget` con la clave `IPC_PRIVATE`, que permite que el proceso y sus descendientes accedan al segmento, el tamaño necesario (`x` o `y` enteros) y los permisos. Después `shmat` vincula el segmento al proceso y devuelve la dirección con la que se accede al vector como si fuera uno normal.

```
void crear_memoria(int x, int y) {
    shmid_verticales = shmget(IPC_PRIVATE, sizeof(int) * x, IPC_CREAT | 0666);
    pids_verticales = (int *) shmat(shmid_verticales, 0, 0);

    shmid_finales = shmget(IPC_PRIVATE, sizeof(int) * y, IPC_CREAT | 0666);
    pids_finales = (int *) shmat(shmid_finales, 0, 0);
}
```

Llamadas al sistema utilizadas: `shmget(), shmat()`.

5.3.3. Función liberar_memoria()

Qué se necesita. Al terminar el programa hay que desvincular y eliminar la memoria compartida para que no quede reservada en el sistema.

Cómo se ha resuelto y por qué. Primero se desvincula cada vector con `shmdt` y después se elimina cada segmento con `shmctl` y la orden `IPC_RMID`. Solo la ejecuta el superpadre, al final, cuando todos los demás procesos ya han muerto.

```
void liberar_memoria() {
    shmdt((char *) pids_verticales);
    shmdt((char *) pids_finales);
    shmctl(shmid_verticales, IPC_RMID, 0);
    shmctl(shmid_finales, IPC_RMID, 0);
}
```

Llamadas al sistema utilizadas: `shmdt(), shmctl()`.

5.3.4. Función crear_verticales()

Qué se necesita. Crear la cadena de `x` procesos, de forma que cada uno guarde su PID en el vector y ningún padre muera antes que su hijo.

Cómo se ha resuelto y por qué. Un bucle se repite `x` veces y en cada vuelta el proceso actual crea un hijo. El padre espera con `wait` a que ese hijo muera: si no es el superpadre (`i > 0`) termina después con `exit`, y si es el superpadre vuelve a `main` para mostrar su mensaje. El hijo recién creado guarda su PID en la posición `i` del vector y continúa con la vuelta siguiente, creando a su propio hijo. Cuando el último proceso de la cadena sale del bucle, crea los hijos finales llamando a `crear_finales`.

```
void crear_verticales(int x, int y) {
    int i;

    for (i = 0; i < x; i++) {
        if (fork() != 0) {
            wait(NULL);
            if (i > 0) {
                exit(0);
            }
            return;
        }
        pids_verticales[i] = getpid();
    }

    crear_finales(x, y);
    exit(0);
}
```

Llamadas al sistema utilizadas: `fork()`, `wait()`, `getpid()`, `exit()`.

5.3.5. Función crear_finales()

Qué se necesita. Crear los y hijos finales, que cada uno guarde su PID en el vector y muestre su mensaje.

Cómo se ha resuelto y por qué. Un bucle crea y hijos. Cada hijo guarda su PID en su posición del vector de finales, muestra su mensaje con `mostrar_subhijo`, espera 10 segundos con `esperar_alarma` y termina. El proceso que los ha creado espera con `wait` tantas veces como hijos ha creado, de manera que no muere antes que ellos.

```
void crear_finales(int x, int y) {
    int i;

    for (i = 0; i < y; i++) {
        if (fork() == 0) {
            pids_finales[i] = getpid();
            mostrar_subhijo(x);
            esperar_alarma();
            exit(0);
        }
    }

    for (i = 0; i < y; i++) {
        wait(NULL);
    }
}
```

Llamadas al sistema utilizadas: `fork()`, `getpid()`, `wait()`, `exit()`.

5.3.6. Funciones despierta() y esperar_alarma()

Qué se necesita. Los hijos finales deben permanecer vivos unos segundos, sin utilizar `sleep`, para poder comprobar el árbol con `pstree`.

Cómo se ha resuelto y por qué. Se programa una alarma de 10 segundos con `alarm` y el proceso se detiene con `pause`. Como la señal SIGALRM mataría al proceso si no tuviera manejador, se ha

creado la función `despierta`, que está vacía y solo sirve para que la señal despierte al proceso sin matarlo.

```
void despierta(int senyal) {
}

void esperar_alarma() {
    signal(SIGALRM, despierta);
    alarm(10);
    pause();
}
```

Llamadas al sistema utilizadas: `signal(), alarm(), pause()`.

5.3.7. Función mostrar_pids()

Qué se necesita. Tanto el superpadre como los subhijos tienen que mostrar una lista de PID separados por comas.

Cómo se ha resuelto y por qué. Se ha creado una función que recibe un vector y su cantidad de elementos y los escribe separados por comas, sin poner coma después del último, y con un salto de línea final. Así se reutiliza para las dos listas (padres y finales).

```
void mostrar_pids(int *vector, int cantidad) {
    int i;

    for (i = 0; i < cantidad; i++) {
        printf("%d", vector[i]);
        if (i != cantidad - 1) {
            printf(", ");
        }
    }
    printf("\n");
}
```

5.3.8. Funciones mostrar_subhijo() y mostrar_superpadre()

Qué se necesita. Mostrar los dos mensajes que pide el enunciado, uno para los hijos finales y otro para el superpadre.

Cómo se ha resuelto y por qué. `mostrar_subhijo` escribe el PID del proceso y, con `mostrar_pids`, los PID de los `x` procesos de la cadena leídos de la memoria compartida. `mostrar_superpadre` escribe su propio PID y los PID de los y hijos finales, también leídos de la memoria compartida.

```
void mostrar_subhijo(int x) {
    printf("Soy el subhijo %d, mis padres son: ", getpid());
    mostrar_pids(pids_verticales, x);
}

void mostrar_superpadre(int y) {
    printf("Soy el superpadre %d : mis hijos finales son: ", getpid());
    mostrar_pids(pids_finales, y);
}
```

Llamadas al sistema utilizadas: `getpid()`.

5.3.9. Función main()

Qué se necesita. Comprobar los argumentos y coordinar las distintas fases del programa.

Cómo se ha resuelto y por qué. Se comprueba que hay dos argumentos y que ambos son mayores que 0. Después se crea la memoria compartida, se crea el árbol de procesos con `crear_verticales`, se muestra el mensaje del superpadre y se libera la memoria. Las dos últimas acciones solo las realiza el superpadre, porque el resto de procesos terminan dentro de `crear_verticales` y `crear_finales`.

```
int main(int argc, char *argv[]) {
    int x, y;

    if (argc != 3) {
        printf("Error. Uso: %s x y\n", argv[0]);
        return 1;
    }

    x = atoi(argv[1]);
    y = atoi(argv[2]);
    if (x < 1 || y < 1) {
        printf("Error. x e y deben ser mayores que 0\n");
        return 1;
    }

    crear_memoria(x, y);
    crear_verticales(x, y);
    mostrar_superpadre(y);
    liberar_memoria();
    return 0;
}
```

5.4. Compilación y ejecución

```
$ gcc -o hijos hijos.c
$ ./hijos 3 2
```

Con estos argumentos se crea una cadena de 3 procesos de la que cuelgan 2 hijos finales. Cada hijo final muestra el mensaje de subhijo con los PID de sus 3 padres y, cuando todos han terminado, el superpadre muestra su mensaje con los PID de sus 2 hijos finales.